NMAM INSTITUTE OF TECHNOLOGY, NITTE

Department of Information Science and Engineering

OPERATING SYSTEMS

PROGRAMMING ASSIGNMENT REPORT

AI-Assisted Synchronization Algorithms with Program-Generated HTML Simulation

Readers-Writers Problem and Dining Philosophers Problem

(Semaphore-based and Monitor-based implementations in Python)

SUBMITTED BY

Name:	Dhrithi P
USN:	NNM24IS071
Course:	Operating Systems
Github Repository:	https://github.com/Dhrithi2006/OS-Synchronization-Assignment

SUBMITTED TO

Dr Jason Elroy Martis

Associate Professor

Dept Information Science and Engineering, Nitte

1. Problem Statement and Objective

When multiple threads access the same data or resources, unrestricted access will lead to race conditions and incorrect results. This project uses two mechanisms: semaphores and monitors to implement two classic synchronization problems.

Problems implemented are:

Readers-Writers using semaphore

Readers-Writers using a monitor

Dining Philosophers for four philosophers with semaphore

Monitor for four philosophers with dining philosophers

The aim is to show mutual exclusion, synchronization, deadlock avoidance and starvation. Each of the four programs logs its events during execution and produces an HTML-based simulation after the real execution trac. The OS scheduler decides which thread to run and when, so the behaviour differs between different execution. All programs are implemented in Python with its standard threading library.

2. Readers-Writers - Semaphore Solution

The Readers-Writers problem is about a shared resource such as a database or a file that many threads can use at the time. The main rule is that many Readers can read the shared resource at once. A Writer needs exclusive access to change the shared resource

Shared. Semaphores:

read_count: An integer that counts how many Readers are active.

mutex: A semaphore that is set to 1 and is used to protect the read_count variable when Readers add or remove themselves.

rw_lock (or wrt): A semaphore that is set to 1 and works as the exclusive lock, for the shared resource.

Execution Logic:

When the first Reader arrives the Reader takes the mutex adds one to read_count and takes the rw_lock. By holding rw_lock the Reader stops any Writer from arriving. Later Readers only need to take the mutex to add to read_count and join the first Reader in the critical section. When the last Reader leaves the last Reader subtracts one from read_count to make it zero and

releases the rw_lock, which finally lets a waiting Writer enter. The Writer takes the rw_lock by itself writes to the shared resource. Then releases the rw_lock.

This specific implementation uses a Readers-preference model. Because the rw_lock stays with the Readers as a group a Writer can wait for a long time (starvation) if new Readers keep coming in a continuous stream.

3. Readers-Writers - Monitor Solution

Where semaphores distribute the synchronization among the program, the monitor consolidates the shared resource, the lock and the condition variables into one single well structured entity. As Python lacks an inbuilt OS monitor keyword, the functionality of such is simulated using threading.Lock().

Execution Logic:

The monitor keeps a record of the number of readers present, the presence of a writer and the number of writers waiting to write.

Reader Entry:

Reader tries to enter and read and waits on the condition variable if there is a writer that is writing or any writers waiting to write.

Writer Entry:

Writer will block if there is any reader reading or another writer writing.

In doing this way, we have implemented a writer preference strategy. This resolves the problem of starvation that was there in the semaphore approach. It gives priority to the waiting writers over new readers, but makes the readers suffer from starvation. If the thread is unable to execute itself, it invokes wait() method, which causes it to release the lock and put itself to sleep. It is checked again in a while loop when it wakes up using notify_all().

4. Dining Philosophers (4) - Semaphore Solution

The Dining Philosophers problem demonstrates resource allocation. The problem consists of four philosophers sitting around the dining table. The philosophers need two forks (one on either side) in order to eat. Semaphores are placed for every fork which are initially set to 1.

Asymmetric Approach to Deadlock Prevention:

In the simplest implementation, where every philosopher picks up his left fork and then the right one, deadlock occurs immediately if all the philosophers are hungry at the same millisecond. In order to avoid circular waiting, an asymmetric approach is used.

Philosopher 0, 1, and 2 are directed to pick up the left fork first and then the right fork.

Philosopher 3 is directed to pick up the right fork first and then the left fork.

In other words, by removing the symmetry in the table, the impossibility that all four philosophers can take the first fork and wait for the second one becomes mathematically impossible. At least one philosopher would have been able to obtain the two forks and eat. Although this has nicely solved the deadlock problem, starvation is not completely eliminated owing to the absence of FIFO queue in Python semaphores.

5. Dining Philosophers (4) - Monitor Solution

Monitor Approach:

The monitor implementation solves the problem from a state management point of view. Instead of having philosophers acquire physical forks, the monitor manages the global state of the table based on the philosopher states (THINKING, HUNGRY, EATING).

Execution Logic:

All operations related to checking and acquiring forks are performed atomically by holding a monitor lock.

The philosopher changes his status to HUNGRY and checks the status of his neighbors to the left and right of him.

If one of the neighbors is EATING, he waits on his own condition variable using the wait() function call.

If none of his neighbors are eating, then the philosopher starts eating and acquires both the forks.

Since a philosopher does not hold any one fork while waiting for the other fork, the "holding and waiting" condition, which is very important for deadlock, becomes totally non-existent. Upon finishing, the philosophers become THINKING and invoke notify_all(), or more specifically, notifies only to their neighboring philosophers.

6. Event Recording and HTML Generation

It should be noted that the simulations of visuals included here are not pre-programmed animations but rather dynamic replays of data obtained from the actual working of Python threads.

The Logging Mechanism:

For accurate replaying, there is a specially developed function log_event(), which is only called inside the critical sections of the program. The recorder uses its own threading.Lock() in order to protect the data from being overwritten by other threads. It records:

An accurate timestamp (millisecond precision).

The identification number of the executing thread (Reader, Writer, or Philosopher).

Current state of the acting thread.

Exact state of ownership of the resources (which philosopher owns Fork 2, for example).

HTML & SVG Drawing:

Once all threads are done executing, the JavaScript converts this event list to JSON and embeds it into an HTML template.

For the Readers-Writers visualization, the JavaScript updates the DOM to depict the movement of the threads between "Waiting," "Reading," and "Writing" queues.

In the Dining Philosophers visualization, the visualizer employs a physical circular layout. With exact mathematical positioning based on the array in the backend, the JavaScript

generates dynamic SVG to create glowing lines of connections between the philosophers and their respective forks.

7. Execution Traces

As the programs depend on concurrent execution with random scheduling delays, the exact order of execution differs in each run.Below is presented the example of the execution process.

7.1 Readers-writers Using Semaphore

This trace shows several readers accessing the resource at once, and writers are required to wait until all the readers leave.

0.018s R2 -> waiting

0.018s R2 -> reading (value=0) // R2 enters critical section

0.022s R1 -> waiting

0.022s R1 -> reading (value=0) // R1 enters simultaneously (Readers Preference)

0.076s W2 -> waiting // W2 arrives but is blocked by rw_lock

0.118s R2 -> idle // R2 leaves

0.122s R1 -> idle // R1 leaves, rw_lock is released

0.122s W2 -> writing (value 0->1) // W2 finally acquires exclusive access

7.2 Dining Philosophers Using Monitors

In this trace, we see the “all-or-nothing” property. The philosopher goes into the hungry state but only acquires the resources after both are available.

0.025s P3 -> hungry

0.026s P3 -> acquiring

0.026s Fork F3 taken by P3 // Acquired simultaneously

0.026s Fork F0 taken by P3 // Acquired simultaneously

0.026s P3 -> eating

0.126s P3 -> releasing

0.126s Fork F3 released

0.126s Fork F0 released

8. Race Conditions, Mutual Exclusion, Deadlock and Starvation

Race Condition:

When more than one thread reads from and writes into shared variables without adequate synchronization, the process results in a race condition. If, for instance, two writers simultaneously get the value of zero and add 1 to it, the end result is 1 and not 2 (loss of update). In our implementation of the program, we are able to solve this problem with the use of semaphores and condition variables.

Mutual Exclusion:

The mutual exclusion principle dictates that only one thread accesses the critical section at any time. This is ensured in our code in that only one writer can modify the shared variable at any one time, and one fork only belongs to one philosopher. This is accomplished through the use of rw_lock in the semaphore code, and monitor: lock in the monitor code.

Deadlock:

For a deadlock situation to exist, the following four conditions must all be present at once: mutual exclusion, hold-and-wait, no preemption, and circular waiting.

In the Semaphore approach, deadlock is avoided since circular waiting is broken through the asymmetric fork acquisition strategy.

In the Monitor approach, deadlock is avoided since hold-and-wait is broken because philosophers examine and obtain both forks at once.

Starvation:

Starvation is where the waiting time of a thread goes on infinitely since the scheduling strategies always give preference to other threads.

The reader-writer semaphore strategy has the readers' preference policy; therefore, writers are bound to suffer from starvation if more readers come.

The strategy of monitor has writers' preference policy that leads to starvation of readers.

In the dining philosophers problem, although deadlock is mathematically impossible, starvation is possible for a philosopher sitting between two philosophers who eat alternately infinitely.

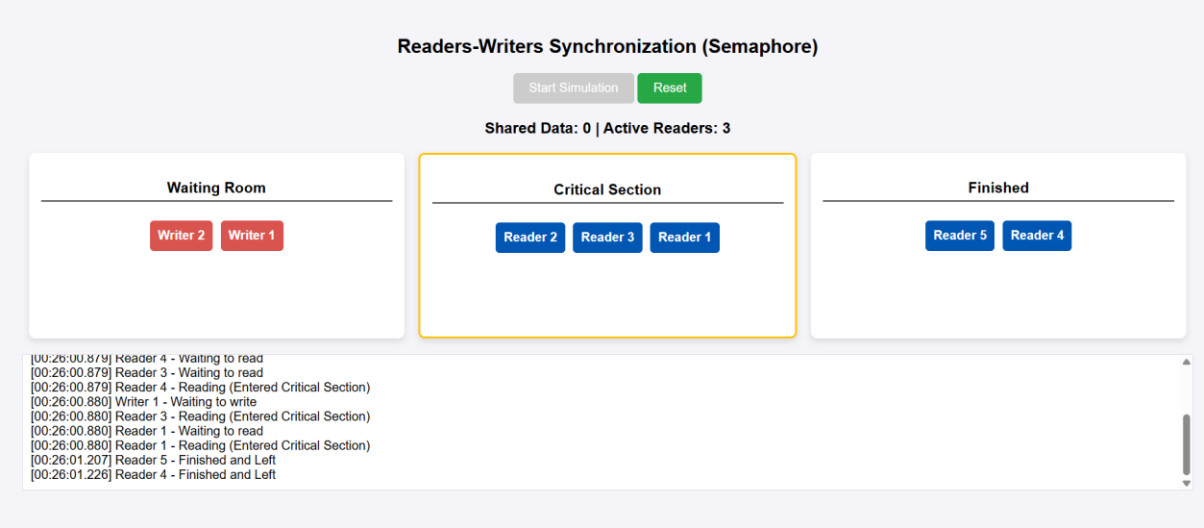
9. Semaphore vs Monitor Comparison

Aspect	Semaphore	Monitor
Structure	Uses primitive integer counters (semaphores) distributed across functions.	Combines shared data, a lock, and condition variables into one encapsulated class.
Waiting Mechanism	Threads block aggressively using acquire().	Threads wait cooperatively using condition variables (wait()).
Resource Control	The programmer is explicitly responsible for placing acquire() and release() operations in the correct spots.	Resource access is controlled strictly inside monitor methods, executing atomically.
Error Risk	Extremely high. Incorrect acquire()/release() ordering, or forgetting a release, causes catastrophic synchronization failure (deadlock).	Lower risk. Encapsulation makes synchronization easier to organize and verify.

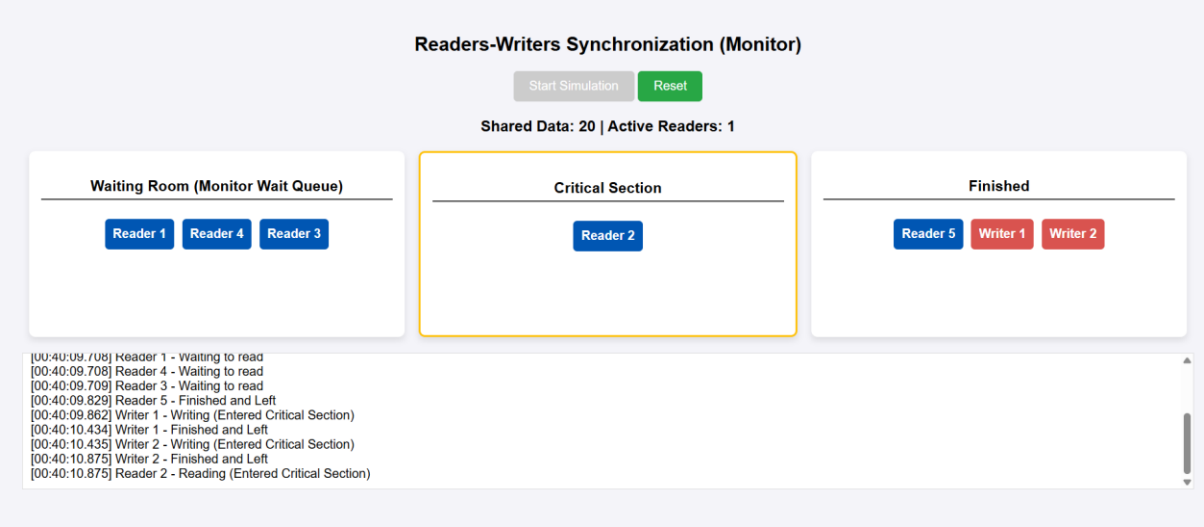
Conclusion: Semaphores offer great flexibility but put all of the responsibility on the programmer. Monitors combine data, locks, and code in one single piece, which makes it much easier to ensure that the rules of synchronization are satisfied and prevents basic order errors.

10. Screenshots of Generated HTML Simulations

10.1 Readers-Writers Using Semaphore:



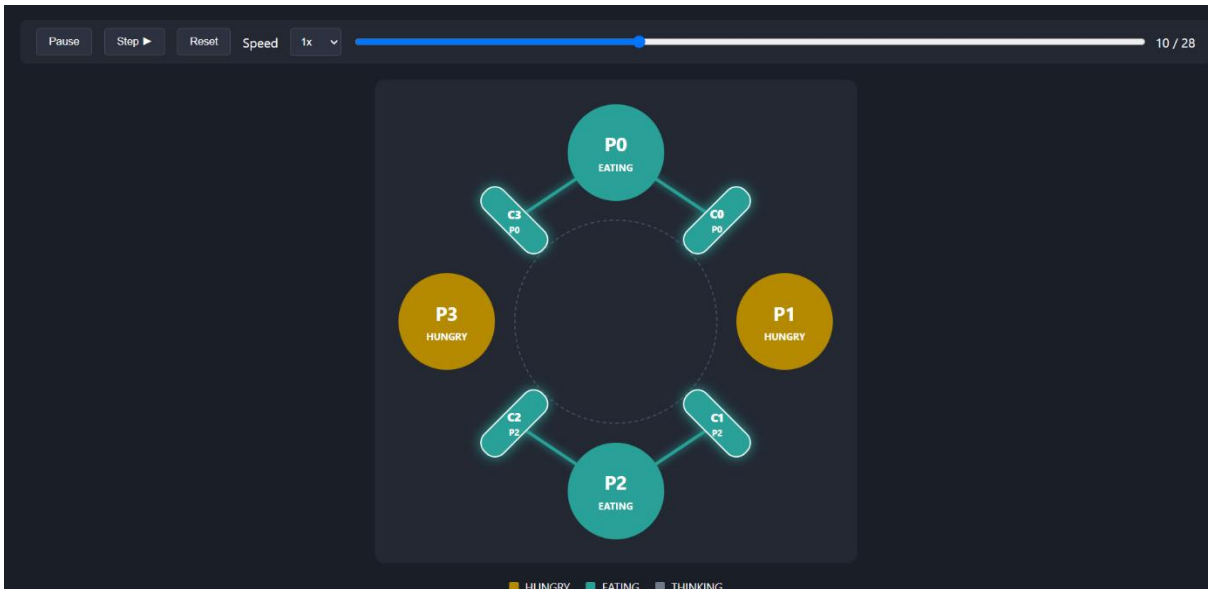
10.2 Readers-Writers Using Monitors:



10.3 Dining Philosophers Using Semaphore:



10.4 Dining Philosophers Using Monitor:



11. AI Usage Summary

Google Gemini was utilized during the development phase for planning, programming boilerplate thread management, debugging synchronization logic, and creating the HTML/JS visualization dashboard. All of the critical prompts used during the development phase are detailed in ai_prompts/prompts.md.

These programs were run several times to ensure that the synchronization behaved as expected based on theory. One major limitation found during development was the issue of spatial

awareness of the AI. Although the AI was able to properly code the backend semaphore logic, it was unable to accurately translate the logical array indices into the physical layout of the SVG in the UI (resulting in improper drawing of the lines through the table). The human was necessary to connect the backend mathematical logic to the frontend physical logic. Moreover, it was tested that although Python condition variables protect against deadlock, they do not guarantee FIFO behavior, which means starvation is theoretically possible.

12. References

[1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.

[2] Tanenbaum, A. S., & Bos, H. (2014). *Modern Operating Systems* (4th ed.). Pearson.

[3] Dijkstra, E. W. (1965). *Co-operating sequential processes*. Technological University Eindhoven, EWD123.

[4] Hoare, C. A. R. (1974). *Monitors: An operating system structuring concept*, Communications of the ACM, vol. 17, no. 10, pp. 549-557.

[5] Python Software Foundation, "threading - Thread-based parallelism," Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/threading.html

[6] MDN Web Docs. (n.d.). *SVG: Scalable Vector Graphics*. Mozilla. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/SVG

[7] Google, "Gemini," AI assistant.

13. Academic Integrity Declaration

I declare that I have personally tested and understood the submitted programs. I have disclosed the AI tools and external resources used in developing this assignment. I can explain the synchronization logic, generated HTML simulation, and modifications made to the code. I have not knowingly submitted another student's work or unacknowledged third-party material as my own.

Name: DHRITHI P

USN: NNM24IS071

Signature: 

Date: 13/10/2026